

Forward Propagation in Python (From Scratch)

The goal of this exercise is to build intuition about what TensorFlow is doing "under the hood." By implementing the calculations manually, you can understand the mechanics of a neural network.

Note: This implementation will use **1D NumPy arrays** (e.g., `np.array([...])`) for vectors and parameters, unlike the 2D matrices (e.g., `np.array([[...]])`) expected by TensorFlow's `Sequential` model. This simplifies the from-scratch code.

Coffee Roasting Example Recap

- **Input ($\vec{x}$):** 2 features
 - **Layer 1 ($\vec{a}^{[1]}$):** 3 neurons
 - **Layer 2 ($\vec{a}^{[2]}$):** 1 neuron (output)
-

Implementation Steps 🧑🏻💻

We will compute the activation for each neuron, one by one.

1. Compute Layer 1 Activations

- **Input:** `x` (a 1D NumPy array with 2 features)
- **Neuron 1 ($a_1^{[1]}$):**
 1. Define parameters: `w1_1` (vector), `b1_1` (scalar)
 2. Compute `z`: `z1_1 = np.dot(w1_1, x) + b1_1`
 3. Apply activation: `a1_1 = sigmoid(z1_1)`
- **Neuron 2 ($a_2^{[1]}$):**
 1. Define parameters: `w1_2`, `b1_2`
 2. Compute `z`: `z1_2 = np.dot(w1_2, x) + b1_2`
 3. Apply activation: `a1_2 = sigmoid(z1_2)`
- **Neuron 3 ($a_3^{[1]}$):**
 1. Define parameters: `w1_3`, `b1_3`
 2. Compute `z`: `z1_3 = np.dot(w1_3, x) + b1_3`
 3. Apply activation: `a1_3 = sigmoid(z1_3)`
- **Collect Activations:**
 - Group the individual neuron outputs into a single vector, which is the output of

Layer 1.

- `a1 = np.array([a1_1, a1_2, a1_3])`

2. Compute Layer 2 (Output Layer) Activation

- **Input:** `a1` (the 1D NumPy array with 3 activations from Layer 1)
- **Neuron 1 ($a_1^{[2]}$):**
 1. Define parameters: `w2_1` (vector), `b2_1` (scalar)
 2. Compute z : `z2_1 = np.dot(w2_1, a1) + b2_1`
 3. Apply activation: `a2_1 = sigmoid(z2_1)`
- **Final Prediction:** The value `a2_1` is the final output (probability) of the network.

This manual, neuron-by-neuron process is exactly what a framework like TensorFlow does for you, but in a highly optimized, vectorized way.